

Preparazione esame – UnityGameSDTRA

Domande prevedibili dal professore e dove modificare il codice — Sistemi per la Riabilitazione e la Terapia Assistita

Il professore tipicamente fa **modifiche live** per vedere se sai *dove* mettere le mani. Le domande qui sotto sono organizzate per difficoltà crescente, con il riferimento esatto al file e alla riga da modificare.

Livello 1 — Modifiche banali (quasi certe)

Sono quasi tutte in `Assets/Codice/Impostazioni.cs` — è il file da aprire per primo.

Domanda	Dove modificare
"Cambia il numero di vite a 5"	<code>VITE = 3 → 5</code>
"Dai 90 secondi per livello"	<code>TEMPO_LIVELLO = 60f → 90f</code>
"Una caramella deve valere 25 punti"	<code>PUNTI_CARAMELLA = 10 → 25</code>
"Quanti punti si perdono se tocchi un asteroide? Raddoppialo"	<code>PUNTI_PERSI_HIT = 5 → 10</code>
"Rendi più facile prendere le caramelle"	aumenta <code>RAGGIO_CARAMELLA</code>
"Nel livello 1 ci devono essere 20 caramelle"	<code>CARAMELLE_LIV1 = 10 → 20</code>
"Allunga il PRONTI iniziale a 3 secondi"	<code>TEMPO_PRONTI = 1.5f → 3f</code>

Trappola: se cambia solo la `const` ma non vede effetto, gli serve **ricompilare** (basta tornare in Unity e aspettare). Spiegalo se chiede "perché non funziona".

Livello 2 — Colori e aspetto

Quasi tutto in `Assets/Codice/FabbricaImmagini.cs`.

Domanda	Dove
"Fai Astro blu invece di verde"	<code>CreaAstro()</code> → cambia <code>pelle = new Color(0.40f, 0.85f, 0.40f)</code> in qualcosa tipo <code>new Color(0.30f, 0.50f, 1f)</code>
"Cambia il colore dell'antenna in rosso"	<code>antenna = new Color(1f, 0.85f, 0.20f)</code>
"Cambia lo sfondo da viola scuro a nero"	<code>Avvio.cs:30</code> — <code>c.backgroundColor</code>
"Voglio meno stelle nello sfondo"	<code>CaricatoreLivelli.cs:77</code> — <code>for (int i = 0; i < 120; i++)</code> → cambia il 120
"Cambia il colore della barra energia"	<code>InterfacciaGioco.cs:69</code> — <code>texRiempimentoBarra</code>
"Cuori più grandi"	<code>InterfacciaGioco.cs:164</code> — <code>stileCuori.fontSize</code>

Livello 3 — Logica di gioco semplice

Domanda	Dove
"Quando tocchi un asteroide perdi 2 vite invece di 1"	<code>GestoreGioco.cs:270</code> <code>Vite - 1</code> → <code>Vite - 2</code>
"L'allarme tempo deve partire a 15 secondi, non 10"	<code>InterfacciaGioco.cs:214</code> — <code>gm.TempoRimasto</code> <code><= 10f</code>
"Astro respira più velocemente"	<code>Astro.cs:73</code> — <code>Time.time * 2f</code>
"La caramella deve oscillare di più"	<code>Caramella.cs:52-53</code> — <code>i</code> <code>0.10f</code> e <code>0.12f</code>
"Il cooldown dopo un colpo dura 2 secondi invece di 1"	<code>GestoreGioco.cs:237</code> <code>cooldownDanno = 1.0f</code> → <code>2.0f</code>
"Astro deve essere più grande"	<code>CaricatoreLivelli.cs:146</code> — <code>localScale</code> <code>1.5f</code>

Livello 4 — Modifiche concettuali (qui vede se hai capito davvero)

Domanda	Cosa richiede
"Quando perdi una vita, fai ripartire il tempo da 30 secondi invece che da 60"	In <code>PerdiUnaVita</code> o <code>TempoScaduto</code> in <code>GestoreGioco.cs</code> aggiungere <code>TempoRimasto = 30f;</code>
"Aggiungi un bonus: ogni 5 caramelle, +1 vita"	In <code>SegnalaCaramellaRaccolta</code> controllo <code>if (CaramelleRaccolte % 5 == 0) Vite++;</code>
"Se finisci con più di 20 secondi rimasti, +50 punti bonus"	In <code>SegnalaPortaRaggiunta</code> , prima di <code>MissioneCompletata = true</code>
"Le bombe non devono muoversi"	<code>Bomba.cs:65-69</code> — togliere o commentare il calcolo <code>dx/dy</code>
"La porta si deve aprire anche senza chiave"	<code>GestoreGioco.cs:221</code> — togliere <code>if (!ChiavePresata) return;</code>
"Fai partire il gioco dal livello 2"	<code>GestoreGioco.cs:96</code> — <code>CaricaLivello(0)</code> → <code>CaricaLivello(1)</code>
"Togli il MENU"	<code>InterfacciaGioco.cs:105-109</code> commentare il bottone
"Il flash rosso dello schermo deve essere blu"	<code>InterfacciaGioco.cs:88</code> — il <code>new Color(1f, 0.20f, 0.20f, ...)</code>

Livello 5 — Domande concettuali a voce (dove fai i punti)

1. "Perché usi `static List<Caramella> Attive` invece di cercare le caramelle ogni volta?"

→ **Performance:** `FindObjectByType` è $O(n)$ sull'intera scena ogni frame. Mantenere una lista aggiornata in `OnEnable` / `OnDisable` è $O(1)$.

2. "Cosa succede se in `ControllaCaramelle` itero `for (int i = 0; i < Caramella.Active.Count; i++)` invece che al contrario?"

→ `Raccogli()` chiama `Destroy(gameObject)` che fa `OnDisable` → la caramella si toglie dalla lista mentre ci sto iterando → si saltano elementi o eccezione. Itero al contrario per evitare il problema.

3. "Perché `Impostazioni` è `static class` e non `MonoBehaviour`?"

→ Contiene solo costanti di configurazione, non ha stato, non deve stare nella scena. Una `static class` è più leggera e accessibile da ovunque senza riferimenti.

4. "Perché c'è il `cooldownDanno` ?"

→ Senza, restando "incollato" a un asteroide perderei tutte e 3 le vite in 3 frame consecutivi. Il cooldown di 1 secondo dà il tempo di staccarsi.

5. "Cosa fa `RuntimeInitializeOnLoadMethod` in `Avvio.cs`?"

→ Dice a Unity di chiamare quel metodo automaticamente all'avvio della scena, senza bisogno che lo script sia attaccato a un GameObject. Così la scena può essere vuota.

6. "Perché `Astro.Istanza` è statico?"

→ **Singleton pattern** — c'è un solo Astro in scena, lo rendo accessibile da chiunque senza dover passare riferimenti.

7. "Cos'è `Time.deltaTime` e perché lo moltiplichiamo nei timer?"

→ È il tempo trascorso dall'ultimo frame. Moltiplicandolo (o sottraendolo) i tempi diventano indipendenti dal frame rate.

8. "Perché in `Astro.cs:69` fai `Mathf.Max(Time.deltaTime, 0.000001f)` ?"

→ Per evitare divisione per zero nel calcolo della velocità.

9. "Cosa cambia se metto `Update` o `FixedUpdate` ?"

→ `Update` gira a ogni frame (frame rate variabile), `FixedUpdate` a intervalli fissi (usato per la fisica). Qui non uso fisica Unity quindi `Update` va bene.

10. "Cos'è `sortingOrder` ?"

→ Ordine di disegno degli sprite 2D — chi ha valore più alto sta davanti. Astro ha 10 per stare sempre sopra a tutto.

Domanda "killer" da preparare bene

Quasi sicuro te la fa: "Spiegami il flusso completo da quando premo Play a quando appare Astro."

Risposta strutturata:

1. Unity carica la scena vuota `SceneAvvio.unity`
2. `RuntimeInitializeOnLoadMethod` chiama `Avvio.Inizia()` automaticamente
3. `Avvio` crea la telecamera, un `GestoreGioco` e una `InterfacciaGioco`
4. `GestoreGioco.Start()` chiama `CaricaLivello(0)`
5. `CaricaLivello` legge i dati da `DefinizioneLivelli.Ottieni(0)` (struttura `DatiLivello`)
6. Chiama `CaricatoreLivelli.Carica(dati)` che costruisce stelle, pianeti, **Astro**, asteroidi, bombe, caramelle
7. Da quel momento `Astro.Update()` segue il mouse, `GestoreGioco.Update()` fa scorrere il tempo

Consigli operativi durante l'esame

1. **Tieni aperto Unity** durante l'esame, non solo l'editor — così ogni modifica la *testi al volo* premendo Play.
2. **Apri prima `Impostazioni.cs` e `Avvio.cs`** appena ti dà la domanda: il 70% delle modifiche sta lì o nei file linkati da lì.
3. **Se ti chiede una cosa che non ricordi dove sta**, usa `Cmd+Shift+F` in VS Code per cercare nel progetto (es. cerca `VITE` o `60f` o `Color`).
4. **Se sbagli e Unity dà errore**: non agitarti, leggi il messaggio in console, quasi sempre ti dice file e riga.